

Building a Chat Interface for a Web App with a Custom LLM

Agenda

1. What we are building

The product, the interface, and the architecture behind it.

2. Where we are going

From a small hand-written tool catalogue to real, complex production APIs.

3. Data

Sourcing, formatting, preprocessing and validating tool-calling data.

Every part ends with a **reading checklist** – the keywords and links to work through before you start on that part.

4. Training

Fine-tuning a small model: PEFT, LoRA/QLoRA, and cross-validation so the number you report is real.

5. Evaluation metrics

What “accuracy” means for tool-calling, and how to test it.

6. Serving

Getting the trained model behind an API.

PART 1

What we are building

The objective

One sentence, and the problem hiding underneath it

Goal: give a web application a chat interface, so a user can ask a question in natural language – *“how many laptops are still under warranty?”* – instead of clicking through six screens.

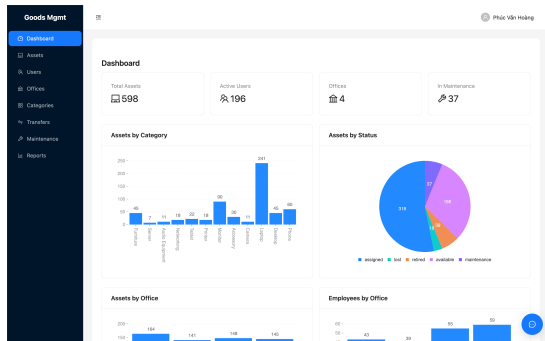
Constraint: do it with a **small model we own and serve ourselves**, not only a frontier API model.

Why this is a **tool-calling** problem, not a chatbot problem:

- The model must never *invent* an answer. It picks the right API, fills the right arguments, and lets real backend data speak.
- So the unit of correctness is **correct tool + correct parameters + correct permission** – not fluent prose.
- Which means it is measurable. That is what makes it engineering rather than guesswork.

The application it plugs into

An enterprise asset-management web app: companies, offices, users, assets, transfers, maintenance



Dashboard

Assets

Search: [] Status: [] Category: []

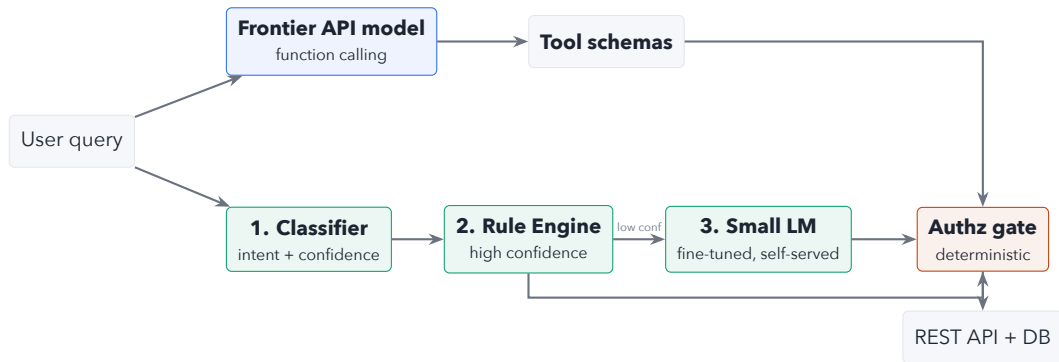
Asset Tag	Name	Category	Office	Assigned To	Status	Condition	Purchase Cost
ACME-AU-00011	Jabra Speak 750	Audio Equipment	Chi nhánh Đà Nẵng	Nguyễn Văn An	ASSIGNED	GOOD	7,834,000 VND
ACME-AU-00010	Jabra Speak 750	Audio Equipment	Chi nhánh TP.HCM	Nguyễn Văn An	ASSIGNED	FAIR	7,523,000 VND
ACME-AU-00008	Poly Studio P15	Audio Equipment	Chi nhánh TP.HCM	Nguyễn Văn An	ASSIGNED	FAIR	12,997,000 VND

Total 3 assets < 1 >

Asset list – the same data the chat tools read

Stack: a **REST API** (Flask + flask-restx), **PostgreSQL**, JWT auth, and a **React + TypeScript** frontend. The chat widget is just one more client of that same API – it has no privileged path to the database.

Two modes, one tool catalogue



Baseline mode – every query goes to a frontier API model. **Cascade mode** – three tiers, cheapest first, and **no fallback to the API model**, so the two modes stay independently comparable.

The chat widget

Same query, same tool call, same conclusion – the two modes are interchangeable to the user

×

Trợ lý AI

Hybrid (Rule Engine + SLM)

GPT-5.2 Only (Baseline)

Xin chào! Tôi là trợ lý AI của Hệ thống Quản lý Tài sản. Bạn cần hỗ trợ gì?

Tôi muốn hủy yêu cầu điều chuyển đang chờ của mình.

Hiện tại bạn **không** có yêu cầu điều chuyển nào ở trạng thái "pending" để hủy.
Nếu bạn có **mã yêu cầu** (UUID), gửi cho tôi để tôi hủy trực tiếp theo mã.

GPT-5.2

🕒 5897 ms

list_transfers

Hỏi về tài sản, nhân viên, văn phòng...

➤

Baseline mode – frontier API model

×

Trợ lý AI

Hybrid (Rule Engine + SLM)

GPT-5.2 Only (Baseline)

Xin chào! Tôi là trợ lý AI của Hệ thống Quản lý Tài sản. Bạn cần hỗ trợ gì?

Tôi muốn hủy yêu cầu điều chuyển đang chờ của mình.

Bạn hiện không có yêu cầu điều chuyển nào đang chờ duyệt.

Rule Engine

🕒 10 ms

list_transfers

Hỏi về tài sản, nhân viên, văn phòng...

➤

Cascade mode – handled by the rule tier

Permissions are decided by code, never by the model

× Trợ lý AI 🏠

Hybrid (Rule Engine + SLM) GPT-5.2 Only (Baseline)

Xin chào! Tôi là trợ lý AI của Hệ thống Quản lý Tài sản. Bạn cần hỗ trợ gì?

Tôi cần thay đổi trạng thái tài sản AS-117, nhờ xét duyệt cho tôi quyền này.

Bạn **không có quyền** tự thay đổi trạng thái tài sản hoặc xin cấp quyền trực tiếp qua hệ thống chat này (vai trò hiện tại của bạn là **Nhân viên**).

Bạn vui lòng:

- Gửi yêu cầu tới **Quản trị viên công ty** hoặc **IT/Asset Admin** để được **cấp quyền cập nhật tài sản** (update/đổi trạng thái).
- Cung cấp kèm: **mã tài sản AS-117**, trạng thái muốn đổi sang (available/assigned/maintenance/retired/lost) và lý do.

Nếu bạn cho tôi biết **muốn đổi AS-117 sang trạng thái nào** và lý do, tôi có thể giúp bạn soạn nội dung yêu cầu gửi quản trị viên.

GPT-5.2 5508 ms

Hỏi về tài sản, nhân viên, văn phòng... ➤

Every tool call from *every* tier passes through one fail-closed authorization gate **before** any HTTP request is made.

- One policy table, one row per tool – the only place permission is decided.
- “May execute it” is separate from “is even offered it as a tool”.
- Restricted users get their arguments **force-scoped to self**; whatever the model supplied for those keys is discarded.
- The REST layer re-checks independently. Filtering the tool list is a UX choice, *not* a security boundary.

Read up on this as [OWASP LLM06 – Excessive Agency](#).

A restricted user asking for something above their role

PART 2

Where we are going

The extension: real APIs instead of a small catalogue

This is why you are here

Today

- A few dozen tools, hand-written, flat argument shapes.
- Arguments are mostly scalars: an enum, an ID, a keyword.
- One backend we control, with seeded data.
- The whole catalogue fits in one system prompt.

Next

- Real production APIs: nested objects, arrays, conditional fields, pagination, cursors, date ranges.
- Far more tools than fit in a prompt ⇒ **tool retrieval** becomes its own problem.
- Multi-step: the output of call #1 is an argument of call #2.
- Real error surfaces: 4xx/5xx, rate limits, partial results.

First deliverable: a **new evaluation set on the real APIs**, built and agreed before anyone touches a training recipe. Nothing measured on the simple catalogue can be assumed to carry over.

How the work splits

Data

Owens: turning an API spec into trustworthy training and evaluation rows.

- Tool schema → scenario generation
- Labeling + validation of labels
- Text preprocessing
- Train / val / test splits, leakage control
- Public-dataset replay mixing
- The training row format

Training & Serving

Owens: turning those rows into a served model with a defensible number.

- PEFT / LoRA / QLoRA fine-tuning
- Loss masking, hyperparameters, checkpoints
- Cross-validation and significance testing
- Evaluation metrics and the benchmark harness
- Export and serving

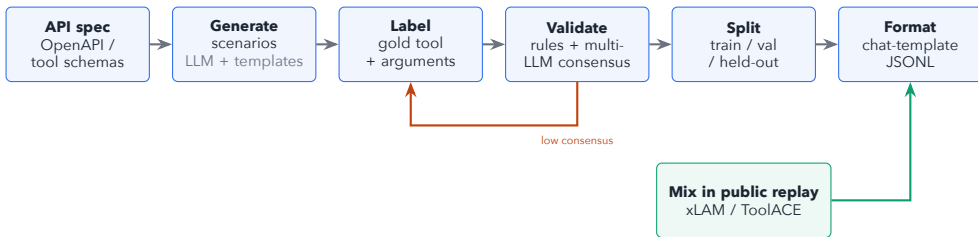
The two meet at exactly two contracts: **the training row format** (Part 3) and **the evaluation set + metric** (Part 5).

PART 3

Data



The data lifecycle



The loop that matters is the orange one: anything the validators disagree on goes back to a human – it does **not** go into the training set.

Data: the keywords

Sourcing

synthetic data generation · **self-instruct** ·
schema-driven generation · **seed prompts**

Label quality

rule-based validation (schema, enums, required args) ·
multi-LLM consensus · **inter-annotator agreement** ·
acceptable alternates for ambiguous queries

Splits

train / val / held-out · **data leakage** · **near-duplicate
detection** · **stratification** by tool and by role

Mixing

replay / rehearsal · **catastrophic forgetting** · **data
mixture ratio** · **dataset licensing**

Preprocessing

chat template · **tokenization** · **word segmentation**
(needed for Vietnamese) · **TF-IDF** char + word n-grams

Vietnamese caveat: do **not** copy English habits – no stop-word lists, no stemming, do not strip diacritics. Function words carry the meaning, and a whitespace token is a syllable, not a word.

The training row format – the contract between the groups

Get this wrong and nothing downstream works

One JSONL line per example. The tool catalogue is rendered by the tokenizer's own `apply_chat_template(tools=...)`, **exactly** as it will be at serving time.

```
{
  "role": "employee",
  "messages": [
    {
      "role": "system",
      "content": "<the exact serve-time system preamble>",
    },
    {
      "role": "user",
      "content": "Tôi được giao những tài sản nào?",
    },
    {
      "role": "assistant",
      "content":
        "<tool_call>{
          \"name\": \"list_assets\",
          \"arguments\": {
            \"assigned_to\": \"usr-001\"
          }
        }</tool_call>"
    }
  ]
}
```

Train/serve parity: whatever the model sees at inference it must have seen in that same shape during training – same system preamble, same tool-rendering path, same `<tool_call>` envelope. This is the most common source of a model that trains beautifully and then fails in production.

Reading checklist – Data

Read before you start on data

- **APIGen** – automated pipeline for verifiable function-calling data ([arXiv:2406.18518](#)) – the model for how to generate and then *verify* tool-calling data.
- **ToolACE** ([arXiv:2409.00920](#)) – data complexity, diversity and accuracy for function calling.
- **ToolLLM / ToolBench** ([arXiv:2307.16789](#)) – how to scale to thousands of real APIs.
- **Function-calling guide** – the tool-schema convention everyone follows.
- **Chat templating docs** – especially the `tools=` argument. Read this one twice.
- **datasets docs** – loading, splitting and mixing.

Public datasets to know – check the licence before using either: [Salesforce/xlam-function-calling-60k](#) (CC-BY-4.0) · [Team-ACE/ToolACE](#) (Apache-2.0)

PART 4

Training

Training: the keywords

You do not need the maths to start – you need to know what these words mean

The model

small language model (SLM) · **instruct-tuned** · **tool-calling chat template** · **context length**

Efficient fine-tuning

PEFT · **LoRA** – rank r , α , target modules, dropout · **QLoRA** · **4-bit NF4 quantisation** · **adapter**

The training run

SFT · **completion-only loss** / **loss masking** · **gradient accumulation** · **warmup** · **weight decay** · **best-checkpoint selection**

Knowing whether it worked

k-fold cross-validation · **StratifiedKfold** / **GroupKfold** · **nested CV** · **multi-seed** · **mean $\pm$ std**

Going further

GRPO and **RLVR** – reinforcement learning with a *programmatic* reward, which suits tool-calling because correctness is checkable in code

Serving

vLLM · **PagedAttention** · **OpenAI-compatible endpoint** · **tool-call parser** · **multi-LoRA** / adapter hot-swap

Two traps worth knowing on day one:

1. Compute loss on the *answer only*. Including a long system prompt in the loss drowns the signal.
2. If the max sequence length is shorter than the prompt, the answer is truncated away, the loss mask covers nothing, and training **silently** does nothing.

Cross-validation: why k-fold, and where it applies

Part of training, not an afterthought – it is how you know a recipe is really better

A single train/test split gives you *one* number with no error bar. With a few thousand rows, that number moves by several points depending only on which examples landed in which split. **k-fold cross-validation** divides the data into k parts, trains k times leaving one part out each time, and reports **mean $\pm$ standard deviation**.

Fold 1	validate	train	train	train
Fold 2	train	validate	train	train
Fold 3	train	train	validate	train
Fold 4	train	train	train	validate

Two places it applies: **(a)** the intent classifier – cheap, so cross-validate by default; **(b)** the fine-tuned model – expensive, since each fold is a full training run, so budget for it or use multi-seed instead. Either way, the metrics in Part 5 are what you compute inside each fold.

k-fold: the variants you should know

KFold – the plain version.

StratifiedKFold – keeps the class balance in every fold. **Use this** when intents or tools are unevenly represented, which they always are.

GroupKFold – keeps related rows together. Essential when several paraphrases came from one template: otherwise a paraphrase of a training row lands in validation and the score is inflated.

Nested CV – an inner loop for hyperparameters, an outer loop for the estimate. The honest way to tune and report at the same time.

For fine-tuning specifically

Full k-fold means k training runs. Two cheaper alternatives in common use:

- **multi-seed** – same split, n random seeds, report mean $\pm$ std. Catches run-to-run variance, not split variance.
- **repeated hold-out** – resample the split a few times.

Whichever you pick, **say which one you used** when you report the number.

The **final** evaluation set stays untouched by all of this. Cross-validate *inside* the training data only.

Reading checklist – Training

Read before you start on training

- **LoRA** ([arXiv:2106.09685](#)) – read the method section properly; everything else builds on it.
- **QLoRA** ([arXiv:2305.14314](#)) – 4-bit NF4, double quantisation, paged optimisers.
- **PEFT docs** and **TRL SFTTrainer** – the practical path. Look specifically for `completion_only_loss`.
- **GRPO / DeepSeekMath** ([arXiv:2402.03300](#)) with **TRL GRPOTrainer** – for once supervised fine-tuning has plateaued.
- **Qwen2.5 technical report** ([arXiv:2412.15115](#)) – the model family we use and its tool-calling support.
- **FrugalGPT** ([arXiv:2305.05176](#)) / **RouteLLM** ([arXiv:2404.14618](#)) – the cascade/routing argument the architecture rests on.
- **scikit-learn – Cross-validation** – KFold, StratifiedKFold, GroupKFold, `cross_val_score`. The canonical practical explanation.
- **scikit-learn – Common pitfalls** – data leakage, and why preprocessing is fitted inside the fold.

Tooling to have used at least once: [peft](#) · [trl](#) · [bitsandbytes](#) · [unsloth](#) · [vLLM](#)

PART 5

Evaluation metrics

What “accuracy” means for tool-calling

Agree the metric before you generate a single training row

Metric	What it answers
Tool-selection accuracy	Did it call the right function at all? The coarsest, most stable signal.
Exact-match / strict	Right function <i>and</i> every argument right. The headline number.
Parameter F1	Partial credit per argument – separates “wrong tool” from “right tool, sloppy arguments”.
AST accuracy	Compares the parsed call structurally rather than as a string. Standard in the public leaderboards.
Executable accuracy	Actually run the call and compare the response. Slow, expensive, closest to truth.
Irrelevance detection	Does it correctly call <i>nothing</i> when no tool applies? The most commonly forgotten one.
Latency p50 / p95	Cost and user experience. Report percentiles, never just the mean.

Also keep a **failure taxonomy** – wrong tool, missing argument, hallucinated argument, malformed JSON, spurious call – because the mix tells you what to fix next.

Reporting a number honestly

Report every metric as a distribution

Run the metric *inside* the cross-validation folds and report **mean $\pm$ std**, not a single value. A metric quoted from one split is an anecdote.

Attach uncertainty

bootstrap confidence interval – resample the test set to get a range rather than a point estimate.

Compare two systems properly

McNemar's test – the right paired test when both systems ran on the *same* items. "A scored two points above B" with no p -value is not a result.

Keep comparisons valid

Any change to the application, the data or the model creates a new measurement baseline. Re-run the benchmark; never quote a number measured under a different configuration.

A workable protocol

1. Freeze a held-out evaluation set. Never train on it, never tune on it.
2. Cross-validate *inside* the training data to choose a recipe – every metric reported as mean $\pm$ std across folds.
3. Run the frozen set once per candidate; report strict accuracy with a confidence interval.
4. Compare candidates pairwise with McNemar.
5. Log every run – date, data version, recipe, result. An unlogged run did not happen.

Reading checklist – Evaluation

Read before you design the evaluation set

- **Berkeley Function-Calling Leaderboard (BFCL)** – the de-facto standard taxonomy: AST vs executable accuracy, multiple and parallel calls, irrelevance detection. Start here.
- **BFCL blog post** – explains each category and exactly how the scoring works.
- **Gorilla** (arXiv:2305.15334) – API correctness and how to measure hallucinated calls.
- **ToolBench / ToolEval** (arXiv:2307.16789) – pass rate and win rate over real API calls.
- `statsmodels` – **McNemar's test** – the implementation you will actually call.
- **scikit-learn – Metrics and scoring** – precision, recall and F1 done right, including averaging modes.
- **scikit-learn – Nested cross-validation** – how to tune and estimate without the estimate becoming optimistic.
- `GroupKFold` **reference** – read this before splitting generated or paraphrased data.

PART 6

Serving



Serving the fine-tuned model

The adapter stays separate from the base model, so you can swap models without redeploying

```
docker run --gpus all -p 8000:8000 vllm/vllm-openai:<tag> \
  --model <base-model> \
  --max-model-len 32768 \
  --enable-auto-tool-choice --tool-call-parser hermes \
  --enable-lora --max-lora-rank 16 \
  --lora-modules my_adapter=/adapters/my_adapter
```

- vLLM exposes an **OpenAI-compatible** /v1/chat/completions, so the backend talks to a self-hosted model and to a commercial API through the same client code.
- The **tool-call parser** turns the model's raw <tool_call> text into a structured tool_calls field. Pick the one matching your model family.
- **Multi-LoRA** serving means several adapters ride on one loaded base model, and the adapter name you register **is** the model name clients ask for – so promoting a new fine-tune is a config change, not a redeploy.

Questions

Work through the reading checklist for your side – data or training – then reproduce one existing result end to end before changing anything.